

Dart programming for Flutter Development

Learn the fundamentals of the Dart programming language before building beautiful, cross-platform mobile applications with Flutter.

Complete **BEGINNER** course (2027)

Module	Topic	Concepts Covered 
1	Installing Dart & Setup	Installing Dart SDK, VS Code setup, Running your first Dart program
2	Variables, Data Types & Strings	Variables, Data Types (int, double, String, bool), String manipulation
3	Constants	const and final
4	Type Conversion & User Input	User Input, Type Conversion, String to Integer/Double, Parsing values
5	Operators & Logic	Arithmetic, Assignment, Comparison, Logical Operators, if, else, switch
6	Collections	Lists, Sets, Maps
7	Loops	for, while, do-while, for-in, Loop control (break, continue)
8	Functions	Function declaration, Parameters, Return values, Arrow functions
9	Null Safety	Nullable & Non-nullable types, Null-aware operators
10	Classes & OOP	Classes, Objects, Constructors, Classes Without Initialization, Basic OOP concepts
11	Exception Handling	try, catch, finally, throw
12	Mini Project	FizzBuzz Challenge (Applying loops, conditions, and logic)
13	Conclusion	Course recap and transition to Flutter



DART VARIABLES

- ◆ A variable is a named container used to store data.
- ◆ The **value stored can change** during program execution.
- ◆ Every variable has a name and a value.

Watch on YouTube: <https://youtube.com/Dart>



DART DATATYPES

A data type specifies the kind of value a variable can store.

<u>Data Type</u>	<u>Description</u>	<u>Example</u>
int	Stores whole numbers (integers).	int age = 25;
double	Stores decimal (floating-point) numbers.	double price = 99.99;
String	Stores text or a sequence of characters.	String name = "John";
bool	Stores a Boolean value: either true or false.	bool isLoggedIn = true;
List	Stores an ordered collection of multiple values.	List<String> fruits = ["Apple", "Orange"];
Set	Stores a collection of unique values (no duplicates).	Set<int> numbers = {1, 2, 3};
Map	Stores data as key-value pairs.	Map<String, int> scores = {"John": 90};
dynamic	Can store values of any data type.	dynamic value = "Hello";



DART

STRING MANIPULATION & INTERPOLATION

String Manipulation is the process of performing operations on strings, such as combining, changing, searching, or extracting text.

Watch on YouTube: <https://youtube.com/Dart>



DART CONSTANTS

■ CONST and FINAL

A constant in dart is a variable whose value cannot be changed after it has been assigned.

- ❖ A **final** variable can only be assigned once. After that, its value cannot be changed.
- ❖ A **const** variable is a compile-time constant. Its value must be known before the program runs and cannot be changed.

final	const
You <u>decide not to change the value</u> .	The value is <u>naturally fixed</u> and never changes.
Ex.: final name = "John";	Ex.: const daysInWeek = 7;

Watch on YouTube: <https://youtube.com/Dart>



DART

User Input

User input allows a program to receive information from the user while the program is running.

Like: name, age, email, phone number, etc

ITS RESULT ALWAYS RETURNS STRING

Watch on YouTube: <https://youtube.com/Dart>



DART TYPE CONVERSION

- Type conversion is the process of **changing one data type into another.**

<u>Conversion Function</u>	<u>Purpose</u>
<code>int.parse()</code>	Converts a String to an integer.
<code>double.parse()</code>	Converts a String to a decimal number.
<code>toString()</code>	Converts a value to a String.

Watch on YouTube: <https://youtube.com/Dart>



DART

Type Inference = *var* vs *dynamic*

`var`

Dart automatically determines the data type.

Type is fixed after the first assignment.

Safer and recommended.

`dynamic`

Can store any data type.

Type can change at any time.

Flexible, but should be used only when necessary.

Watch on YouTube: <https://youtube.com/Dart>



DART Operators – arithmetic, assignment, Comparison

<u>Operator Type</u>	<u>Definition</u>	<u>Examples</u>
Arithmetic Operators	Used to perform mathematical calculations on numbers.	+, -, *, /, %
Assignment Operators	Used to assign or update the value of a variable.	=, +=, -=, *=, /=
Comparison Operators	Used to compare two values and return either true or false.	==, !=, >, <, >=, <=

Watch on YouTube: <https://youtube.com/Dart>



DART

Conditional Statement – if and else

A conditional statement allows a program to make decisions based on whether a condition is true or false.

- if
- if...else
- else if
- switch

Watch on YouTube: <https://youtube.com/Dart>



DART

Logical Operators – AND, OR, NOT

Logical operators are used to combine two or more conditions into a single condition.

<u>Operator</u>	<u>Meaning</u>	<u>Description</u>
&&	AND	Both conditions must be true.
	OR	At least one condition must be true.
!	NOT	Reverses the result (true becomes false, false becomes true).

Watch on YouTube: <https://youtube.com/Dart>



DART LOOPS – for loop & while loop

A loop is used to repeat a block of code multiple times without writing the same code repeatedly.

<u>Loop</u>	<u>Description</u>	<u>Best Used When</u>
for Loop	Repeats a block of code a specific number of times.	You know how many times you want to repeat the task.
while Loop	Repeats a block of code as long as a condition is true.	You don't know exactly how many times the loop will run.



DART Increment(++) & Decrement(--)

INCREMENT and DECREMENT operators are used to INCREASE or DECREASE a variable by 1.

<u>Operator</u>	<u>Meaning</u>
++	Increase by 1
--	Decrease by 1



DART functions

A function is a reusable block of code that performs a specific task.

You CALL (invoke) it when you need to use it.

```
void functionName() {  
    // function body  
}
```

Watch on YouTube: <https://youtube.com/Dart>



DART NULL SAFETY - ?

Null safety is a feature in Dart that helps prevent errors caused by using variables that contain no value (null) by making developers explicitly define whether a variable can store null or not.

<u>Operator</u>	<u>Name</u>	<u>One-line Explanation</u>	<u>Example</u>
?	Nullable Type	Allows a variable to store either a value or null.	String? name;
!	Null Assertion Operator	Tells Dart, "Trust me, this value is not null."	print(name!.length);
??	Null-coalescing Operator	Returns the left value if it's not null; otherwise returns the right value.	print(name ?? "Guest");
??=	Null-aware Assignment	Assigns a value only if the variable is currently null.	name ??= "John";
?.	Null-aware Access Operator	Calls a property or method only if the object isn't null.	print(name?.length);



DART NAMED PARAMETER (most important)

Named parameters are **parameters that are identified by their names** instead of their positions.

<u>Positional Parameters</u>	<u>Named Parameters</u>
Depend on the order.	Depend on the parameter name.
Values must be in the correct position.	Values can be written in any order.
Example: <code>greet("John", 20)</code>	Example: <code>greet(name: "John", age: 20)</code>

Watch on YouTube: <https://youtube.com/Dart>